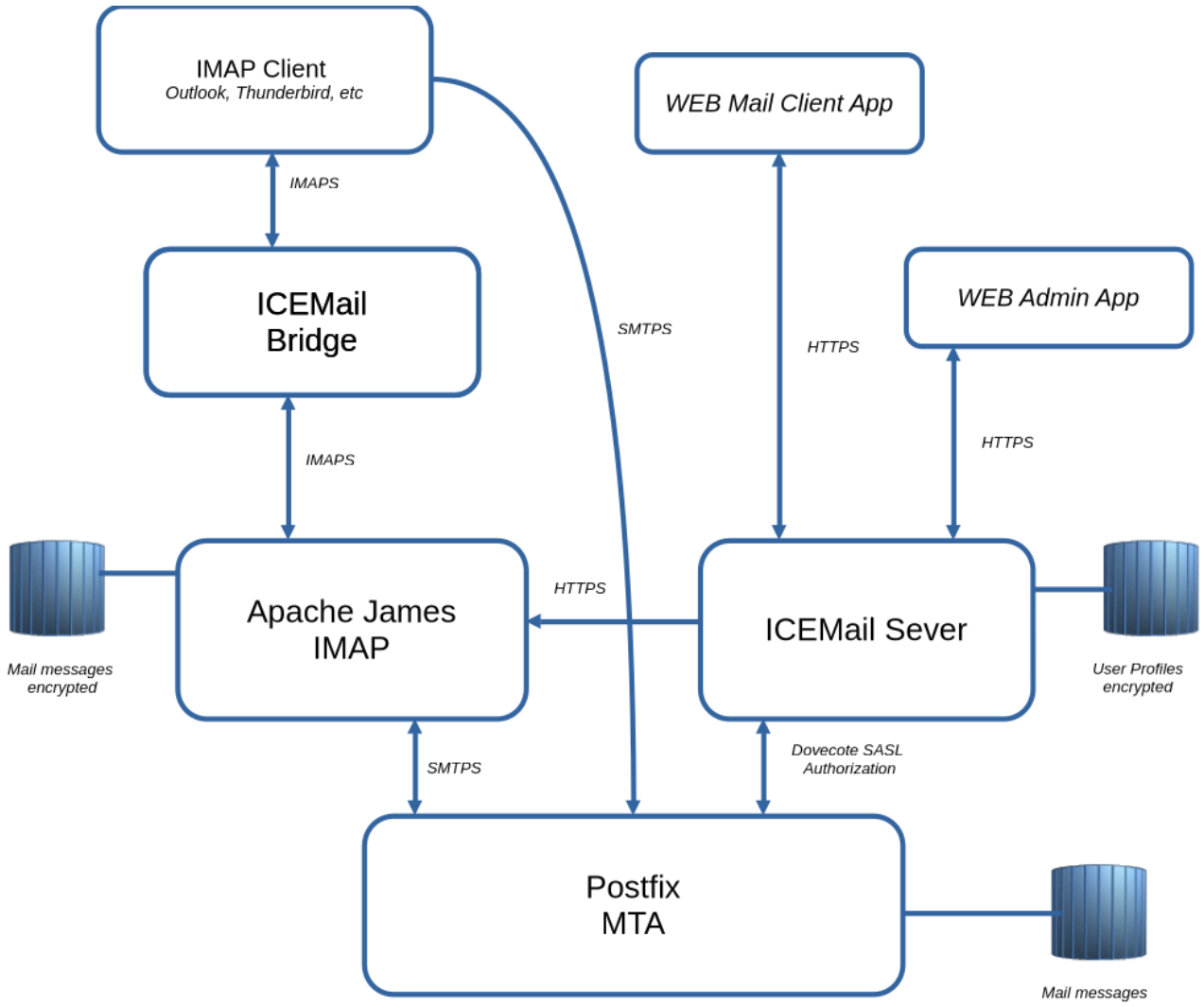
ICEMail — Architecture & Solution Description

Overview

ICEMail is a self-hosted encrypted mail system inspired by Proton Mail. The core philosophy is simple: **all mail content and all user credentials are encrypted at all times, and the servers never have access to plaintext passwords, private keys, or message content.**

Encryption and decryption always happen at the client — either in the browser (web interface) or in the ICEMail Bridge running on the user's own machine. The servers store only ciphertext.



Security Philosophy

Encryption at rest and in transit

Every message stored in the IMAP mailbox is PGP-encrypted with the recipient's public key before it arrives. The IMAP server (James) stores only the encrypted blob. Neither the ICEMail server nor the IMAP server can read any message content.

User profile — what is stored on the server

Each user profile in the ICEMail server database contains:

Field	What is stored
`username`	Plaintext (lowercase)
`password`	PBKDF2 hash (100 000 iterations, SHA-256, salt = username) — computed **in the browser**, never sent in cleartext
`publicKey`	PGP public key in armored format (plaintext — public by design)
`privateKey`	PGP private key, **doubly protected**: first the key itself is protected by a PGP passphrase (the user's password), then the whole armored key is symmetrically encrypted with the password using OpenPGP and stored as Base64
`confirmed`	Account confirmation flag
`lastSeen`	Timestamp

The server never sees the user's plaintext password or the unencrypted private key. PBKDF2 hashing happens in the browser before the login request is sent, and the private key is encrypted in the browser before upload.

Components

The system consists of four main components:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  User's machine                                                              │
│                                                                              │
│  ┌──────────────────┐       ┌──────────────────────────┐                    │
│  │  Mail Client     │       │  Browser (Web App)       │                    │
│  │  (Thunderbird)   │       │  Decryption in JS        │                    │
│  └──────────┬───────┘       └──────────────┬───────────┘                    │
│             │ IMAP/SMTP                    │ HTTPS                          │
│  ┌──────────▼──────────────────────────┐   │                                │
│  │  ICEMail Bridge                     │   │                                │
│  │  IMAP proxy + SMTP proxy            │   │                                │
│  └──────────┬──────────┬──────────────┘   │                                │
└─────────────┼──────────┼──────────────────┼────────────────────────────────┘
              │ IMAPS    │ SMTP             │ HTTPS
┌─────────────┼──────────┼──────────────────┼────────────────────────────────┐
│  Server                                   │                                │
│             │          │                  │                                │
│  ┌──────────▼──────────────────────┐  ┌──────▼──────────────────────────┐  │
│  │  James (IMAP server)            │  │  ICEMail Server                 │  │
│  │                                 │  │  REST API  :8282 (HTTPS)        │  │
│  │  IMAPS      :993  ◄─────────────┼──┼  Web App   :8282 (HTTPS)  (IMAPS)  │
│  │  LMTP        :24  ◄──┐          │  │  After-queue filter :10026/:10027  │
│  │  WebAdmin  :8000  ◄──┼──┼       │  │  SASL server        :12345      │
│  │                      │  │  │    │  │  SQLite3 user DB                │  │
│  │  userSync  :8282 ────┼──┼──►    │  │  /api/users  (HTTPS)            │  │
│  └──────────────────────┘  └──────────────────────────────────────────┘  │
│                          ▲                                                  │
│  ┌──────────────────┐     │                                                │
│  │  Postfix         │     │ SMTP :10026/:10027  after-queue filter         │
│  │  MTA             ├─────┘                                                │
│  │  SMTP :25/:587   │                                                      │
│  │  LMTP ───────────┼──────────────────────────► James :24                 │
│  └──────────────────┘                                                      │
└──────────────────────────────────────────────────────────────────────────────┘
```

---

1. ICEMail Server (`ICE-Server`)

The central server component. It acts as the REST API hub, web application server, user registry, and the point where incoming mail is encrypted.

Source: `ICEMail/ICE-Server`
Technology: Java 21, Javalin (Jetty), SQLite3, BouncyCastle PGP, Log4j2
Default port: 8282 (HTTPS, TLS via PEM cert/key)

1.1 User Profile Database

A SQLite3 database (`ICEMail.sqlite`) holds all user accounts. It stores the PBKDF2-hashed password, the password-encrypted private key, the public key, and account metadata. The server cannot derive the user's actual password or access private key material.

1.2 REST API

The ICEMail Server exposes a REST API over HTTPS used by all other components:

Endpoint	Method	Description
`/register`	POST	Public account self-registration
`/login`	POST	Authenticate and establish a session
`/confirm`	POST	Confirm account via email link
`/api/users`	GET	Returns all user credentials (username + hashed password) — used by James for user sync
`/api/admin`	GET	Returns admin credentials
`/admin/users`	POST	List users (admin session required)
`/admin/handleUser`	POST	Block or delete a user
`/admin/createUser`	POST	Admin-create a confirmed user
`/admin/imap/*`	GET	Proxy to James WebAdmin REST API
`/web/*`	GET/POST	Web mail API (authenticated session)

1.3 Web Interface

A single-page web application served as static files (`WebContent/`) from Javalin. Pages include:

- **`login.html`** — User login
- **`create_account.html`** — Self-registration (if enabled)
- **`confirm.html`** — Account email confirmation
- **`web/index.html`** — Main web mail interface (mailboxes, message list, compose)
- **`web/message.html`** — Individual message view with client-side PGP decryption
- **`decrypt.html`** — Decrypt an ICE-encrypted message sent to an external recipient
- **`admin/index.html`** — Admin panel
- **`admin/users.html`** — User management
- **`admin/imap-*.html`** — IMAP server administration panels (proxied to James WebAdmin)

JavaScript libraries used client-side: `openpgp.js` (PGP operations), `crypto-js` (symmetric crypto), `argon2-bundled` (key derivation).

1.4 Postfix After-Queue Filter

A component of the ICEMail Server module (same Gradle module, same JAR, shares the same `server.json` configuration and SQLite3 database). It has its own `main()` entry point so it is started as a separate process alongside the main server, but it is not an independent component. Postfix routes all inbound mail through it before final delivery. It listens on port **10026** (inbound from Postfix) and re-injects the processed mail back to Postfix on port **10027** — from where Postfix completes normal delivery.

Processing logic:

1. **Loop prevention:** If the message already carries the `X-Filtered-By: JavaEncryptor` header it is re-queued without further processing.
2. **ICE-encrypted message detection:** If the message has an `X-ICE-UID` header (indicating a message composed for an external recipient through the ICE compose-for-external flow), the encrypted payload is extracted, saved to the database, and the message body is replaced with an HTML notification containing a decryption link (`/decrypt.html?uid=...`).
3. **PGP encryption for local recipients:** For each recipient whose address belongs to the configured mail domain, the filter looks up that user's public PGP key in the SQLite3 database. The message body is then **PGP-encrypted with AES-256** using the recipient's public key (via BouncyCastle) and re-queued individually per recipient. `Content-Type` is rewritten to `text/plain`. Recipients outside the domain receive the original unencrypted message.

This is the point where plaintext mail becomes encrypted — it happens transparently between Postfix accepting the message and LMTP delivery to James.

Configuration: Shares the same `server.json` as the main ICEMail Server and reads the SQLite3 database directly.

1.5 SASL Authentication Server (Dovecot SASL protocol)

The ICEMail Server can run an embedded SASL authentication server on a configurable TCP port (default **12345**). Postfix is configured to delegate SMTP AUTH decisions to this server via the Dovecot SASL socket protocol.

When a user submits mail via SMTP (port 587), Postfix asks the SASL server whether the credentials are valid. The SASL server checks the supplied PBKDF2-hashed password against the SQLite3 database.

This allows Postfix to authenticate ICEMail users for SMTP submission without needing its own user database or PAM.

An `internal_mailer` service account (used by the ICEMail Server itself for sending confirmation emails) can also be registered with the SASL server so it can authenticate against the local Postfix SMTP

relay.

1.6 Admin Web Interface

Accessible at `/admin/` (requires admin session). Provides:

- User listing with filter
- Create / block / delete users
- Proxy views into James WebAdmin: domains, mailboxes, mail queues, mail repositories, dead-letter queues

---

2. ICEMail Bridge (`ICE-Mailbridge`)

The Bridge runs on the **user's own machine**. It acts as a local IMAP proxy and optionally an SMTP proxy, sitting between the user's standard mail client (e.g. Thunderbird) and the James IMAP server. Its primary job is transparent PGP decryption of incoming mail.

Source: `ICEMail/ICE-Mailbridge`
Technology: Java 21, pure TCP socket programming, BouncyCastle PGP, Log4j2
Configuration: `mailbridge-<hostname>.json`

2.1 IMAP Proxy

The Bridge listens for IMAP connections from the mail client and connects upstream to the James IMAP server.

Mode	Client-facing port	Server-facing
Plain IMAP	143 (configurable)	James :993 IMAPS
IMAPS	993 (configurable)	James :993 IMAPS

Authentication — password hashing:
The Bridge intercepts the IMAP LOGIN command and transforms the user's cleartext password into its PBKDF2 hash before forwarding to James. This is handled by the `PBKDF2LoginHandler`. The mail client sends the real password; James receives the hash. This ensures the same hashed credential is used everywhere.

Mail decryption — `MailDecryptor`:
As IMAP responses flow back from James to the mail client, the Bridge intercepts them and:
- Detects PGP-encrypted message bodies (`-----BEGIN PGP MESSAGE-----` blocks) and decrypts them

using the user's PGP private key
- Detects encrypted subjects (prefixed `ENC:ICE:<base64>`) and decrypts them

The user's private key and passphrase are supplied to the decryptor at session setup time (obtained from the server after successful login and decrypted locally).

STARTTLS handling:
The Bridge performs a full dual TLS upgrade on STARTTLS requests — it upgrades the connection to the upstream James server first, then upgrades the client-facing connection using the Bridge's own PEM certificate and key.

2.2 SMTP Submission Proxy

When `smtpEnabled: true` in the configuration, the Bridge also listens for SMTP connections from the mail client (default local port **1587**) and proxies them to the upstream Postfix SMTP submission server (port 587).

AUTH transformation:
The Bridge intercepts `AUTH PLAIN` and `AUTH LOGIN` commands, extracts the plaintext password supplied by the mail client, hashes it with PBKDF2 (via the same `LoginHandler` as IMAP), and forwards the hashed credential to Postfix. Postfix then delegates the credential check to the ICEMail Server's SASL server, which compares the hash against the database.

Mail content:
Outgoing mail bodies are forwarded **verbatim** — no server-side encryption is applied to outgoing mail. If the user wants to send encrypted mail to another ICEMail user, that encryption is expected to happen at the mail client level, or the ICEMail compose-for-external feature can be used via the web interface.

TLS:
- The server-side connection (towards Postfix) accepts any TLS certificate (`CustomTrustManager` — trust-all)
- The client-facing STARTTLS upgrade uses the same PEM cert/key as the IMAPS listener

2.3 Configuration

```
{
  "plainEnabled": true,
  "listenPort": 143,
  "imapsEnabled": false,
  "imapsListenPort": 993,
  "imapsCertPath": "cert.pem",
  "imapsKeyPath": "key.pem",
  "imapHost": "mail.example.com",
  "imapPort": 993,
  "loginHandlerClass": "com.hoddmimes.icemail.bridge.PBKDF2LoginHandler",
  "decryptorClass": "com.hoddmimes.icemail.bridge.MailDecryptor",
  "smtpEnabled": true,
  "smtpListenPort": 1587,
  "smtpHost": "mail.example.com",
  "smtpPort": 587,
  "tlsProtocol": "TLSv1.3"
}
```

The configuration file is named `mailbridge-<hostname>.json` so the same set of files can be deployed across multiple machines.

---

3. IMAP Server — Apache James (`james-project`)

Source: `../james-project` (relative to the ICEMail project)
Technology: Forked/enhanced Apache James — a full-featured Java mail server

The James server acts as the IMAP store. It stores mailboxes, delivers and serves messages, and handles all IMAP protocol traffic. It has **no knowledge of the encryption** — from its perspective it is simply storing and serving opaque message content that happens to contain PGP ciphertext.

Ports:

Protocol	Port	Connected by	Notes
IMAPS	993	ICEMail Bridge, ICEMail Server	IMAP over TLS. The Bridge connects on behalf of mail clients; the ICEMail Server connects on behalf of browser sessions (web mail)
LMTP	24	Postfix	Postfix **pushes** encrypted mail to James after the after-queue filter. James listens and stores the message — it does not fetch or pull from Postfix
WebAdmin REST	8000	ICEMail Server	HTTP REST API bound to 127.0.0.1. Used by the ICEMail Server to proxy admin operations (list domains, mailboxes, queues) to the browser
HTTPS user sync	8282 (outbound)	James	James periodically polls the ICEMail Server's `/api/users` endpoint to synchronise user accounts and hashed passwords

User synchronisation:
James does not have its own user registration UI. Instead it periodically polls the ICEMail Server's `/api/users` endpoint to synchronise user accounts (including hashed passwords). This means user creation in ICEMail is automatically reflected in James without manual administration.

Storage:
- User and domain data: Apache Derby embedded database
- Mail index: Apache Lucene

---

4. Postfix (MTA)

Postfix is a standard Linux MTA responsible for receiving inbound SMTP mail from the internet and accepting SMTP submission from authenticated users.

Key configuration points:

Function	Detail
Inbound MX	Standard SMTP on port 25
After-queue filter	All inbound mail is redirected to the ICEMail after-queue filter at port 10026; filtered mail re-enters Postfix at port 10027
Local delivery	Postfix **pushes** filtered (encrypted) mail to James via LMTP on port 24. James listens and accepts — it does **not** fetch or pull from Postfix.
Submission	SMTP AUTH on port 587 with STARTTLS; AUTH delegated to ICEMail Server's SASL server
SASL backend	Dovecot SASL protocol, connecting to ICEMail Server on port 12345

Flow Diagrams

Flow 1 — Inbound Mail Delivery (Internet → Encrypted Mailbox)

```
Internet              Postfix          ICEMail Server             James
                                     (after-queue filter)       (IMAP store)
   |                     |                   |                       |
   |— SMTP :25 ————————▶|                   |                       |
   |  (inbound MX)       |                   |                       |
   |                     |                   |                       |
   |                     |— SMTP :10026 ———▶|                       |
   |                     |  route all inbound |                      |
   |                     |  mail to filter    |                      |
   |                     |                   |                       |
   |                     |    1. Check X-Filtered-By header          |
   |                     |       (loop prevention — already          |
   |                     |        filtered messages pass through)    |
   |                     |                   |                       |
   |                     |    2. For each local recipient:           |
   |                     |       look up public PGP key in SQLite3   |
   |                     |       PGP-encrypt body with AES-256       |
   |                     |       rewrite Content-Type → text/plain   |
   |                     |       re-queue individually per recipient  |
   |                     |                   |                       |
   |                     |    3. For external recipients:            |
   |                     |       pass through unchanged              |
   |                     |                   |                       |
   |                     |◀— SMTP :10027 ———|                       |
   |                     |  re-inject         |                      |
   |                     |  encrypted mail    |                      |
   |                     |  back to Postfix   |                      |
   |                     |                   |                       |
   |                     |— LMTP :24 ————————————————————————▶      |
   |                     |  Postfix PUSHES    |   James listens on   |
   |                     |  (James does not   |   LMTP :24 and       |
   |                     |   fetch/pull)      |   stores PGP         |
   |                     |                   |   ciphertext in      |
   |                     |                   |   the user's mailbox |
```

Key security property: The mail body is PGP-encrypted inside the ICEMail Server before it is handed back to Postfix and before it reaches James. James stores only ciphertext and has no access to the encryption keys. Postfix pushes the encrypted message to James via LMTP — James never retrieves or polls from Postfix.

Flow 2 — Create User Account (Web Interface)

```
Browser                          ICEMail Server                    Postfix/SMTP
   |                                   |                                 |
   |  1. User fills in username,       |                                 |
   |     password, confirmation email  |                                 |
   |                                   |                                 |
   |  2. Browser computes:             |                                 |
   |     - PBKDF2(username, password)  |                                 |
   |       → hashed password           |                                 |
   |     - Generate ECC/Curve25519 PGP |                                 |
   |       key pair (openpgp.js)       |                                 |
   |     - Encrypt private key:        |                                 |
   |         a) PGP key passphrase     |                                 |
   |            = user password        |                                 |
   |         b) Symmetrically encrypt  |                                 |
   |            armored private key    |                                 |
   |            with password → Base64 |                                 |
   |                                   |                                 |
   |——— POST /register ———————————————▶|                                 |
   |  {username, hPassword,            |                                 |
   |   encPrivateKeyB64, publicKey,    |                                 |
   |   confMail, confirmed:false}      |                                 |
   |                                   |                                 |
   |                    Store profile  |                                 |
   |                    in SQLite3     |                                 |
   |                    Generate confUid|                                |
   |                                   |——— Send confirmation email —▶|
   |                                   |     (SMTP submission, STARTTLS)|
   |                                   |                                 |
   |◀——— 200 OK ———————————————————— |                                 |
   |  "Check your email"               |                                 |
   |                                   |                                 |
   |  [User clicks link in email]      |                                 |
   |                                   |                                 |
   |——— POST /confirm {uid} ——————————▶|                                 |
   |                    Mark confirmed |                                 |
   |◀——— 200 OK ———————————————————— |                                 |
```

Key security property: The server only ever receives the PBKDF2 hash of the password and a doubly-encrypted private key. Neither can be used to derive the original password.

Flow 3 — Web App Login

```
Browser                          ICEMail Server
   |                                     |
   |  1. User enters username +          |
   |     password                        |
   |                                     |
   |  2. Browser computes                |
   |     PBKDF2(username, password)      |
   |     → hashed password               |
   |                                     |
   |──── POST /login ──────────────────▶|
   |  {username, hPassword}              |
   |                                     |
   |                    Lookup user      |
   |                    Compare hash     |
   |                    Create session   |
   |◀──── 200 OK + session cookie ───────|
   |                                     |
   |  3. Browser stores cleartext        |
   |     password in sessionStorage      |
   |     (needed for private key         |
   |     decryption later)               |
   |                                     |
   |  4. Navigate to /web/index.html     |
```

Key security property: The server validates only the hash. The cleartext password never leaves the browser. It is kept only in `sessionStorage` for the duration of the browser session, solely to decrypt the private key on demand when reading mail.

Flow 4 — Read Mail via Web Interface (Decryption)

```
Browser                    ICEMail Server              James (IMAP)
   |                              |                          |
   |── GET /web/mailboxes ────────▶|                          |
   |                     IMAP login (hashed pw) ──────────────▶|
   |◀── mailbox list ──────────────|                          |
   |                              |                          |
   |── GET /web/messages ─────────▶|                          |
   |                     IMAP FETCH envelope ─────────────────▶|
   |◀── message list (encrypted) ─|◀──── encrypted mail ────|
   |                              |                          |
   |  [User clicks on a message]  |                          |
   |                              |                          |
   |── GET /web/message ──────────▶|                          |
   |                     IMAP FETCH body ────────────────────▶|
   |◀── encrypted body ────────────|◀──── PGP ciphertext ────|
   |                              |                          |
   |  4. Browser receives the     |                          |
   |     PGP-encrypted body       |                          |
   |                              |                          |
   |── GET /web/profile ──────────▶|                          |
   |◀── encPrivateKeyB64 ──────── |                          |
   |                              |                          |
   |  5. Browser decrypts         |                          |
   |     private key:             |                          |
   |     a) Base64 decode         |                          |
   |     b) OpenPGP symmetric     |                          |
   |        decrypt with password |                          |
   |        from sessionStorage   |                          |
   |     c) Unlock PGP private    |                          |
   |        key with passphrase   |                          |
   |        (= user password)     |                          |
   |                              |                          |
   |  6. Decrypt PGP message body |                          |
   |     using unlocked private   |                          |
   |     key (openpgp.js)         |                          |
   |                              |                          |
   |  7. Display plaintext to user|                          |
```

Key security property: Decryption occurs entirely in the browser. The server delivers the encrypted private key and the encrypted message body. Neither can be decrypted server-side. The plaintext message never traverses the network.

Flow 5 — Compose and Send Mail (Web Interface)

```
Browser                    ICEMail Server        Postfix         James
   |                              |                  |               |
   |  1. User composes message    |                  |               |
   |     (To, Subject, Body)      |                  |               |
   |                              |                  |               |
   |── POST /web/compose/send ──▶|                  |               |
   |   {to, subject, body}        |                  |               |
   |                              |                  |               |
   |             Server sends via SMTP (port 587)    |               |
   |             AUTH with internal mailer account ─▶|               |
   |             STARTTLS, trust-all cert             |               |
   |                              |                  |               |
   |                              |   Postfix receives plain mail     |
   |                              |   Routes through after-queue ─────┤
   |                              |   filter (port 10026)             |
   |                              |                                   |
   |                              |   Filter looks up recipient's     |
   |                              |   public key in SQLite3           |
   |                              |   PGP-encrypts body (AES-256)     |
   |                              |   Re-injects at port 10027        |
   |                              |                                   |
   |                              |   Postfix delivers via LMTP ─────▶|
   |                              |   (port 24, James)                |
   |                              |                  |   Stores PGP   |
   |                              |                  |   ciphertext   |
   |◀── 200 OK ──────────────────|                  |               |
```

Key security property: Even though the web interface sends the message in plaintext to the ICEMail Server over an HTTPS connection, the Postfix after-queue filter immediately encrypts the body with the recipient's public key before the message reaches the mailbox. The server holds the plaintext only for the milliseconds between receiving the compose request and handing it to Postfix.

Flow 6 — Mail Client (Thunderbird) via ICEMail Bridge

```
Thunderbird            ICEMail Bridge            James (IMAP)       ICEMail Server
     |                       |                        |                     |
     |  IMAP LOGIN           |                        |                     |
     |  user / plaintext pw ─►|                        |                     |
     |                       |                        |                     |
     |      1. Bridge hashes password:                |                     |
     |         PBKDF2(username, password)             |                     |
     |                       |                        |                     |
     |         IMAP LOGIN ──►|                        |                     |
     |         user / hash   |─── IMAPS LOGIN ──────►|                     |
     |                       |    user / hash         |                     |
     |                       |◄── OK ────────────────|                     |
     |◄── OK ────────────────|                        |                     |
     |                       |                        |                     |
     |  [Thunderbird fetches |                        |                     |
     |   message list]       |                        |                     |
     |                       |                        |                     |
     |  SELECT INBOX ───────►|─── SELECT INBOX ─────►|                     |
     |◄── EXISTS / FLAGS ────|◄── EXISTS / FLAGS ────|                     |
     |                       |                        |                     |
     |  FETCH ... ──────────►|─── FETCH ... ────────►|                     |
     |                       |◄── PGP ciphertext ────|                     |
     |                       |                        |                     |
     |      2. Bridge decrypts message body:          |                     |
     |         - Detects -----BEGIN PGP MESSAGE-----  |                     |
     |         - Decrypts using user's private key    |                     |
     |         - Replaces ciphertext with plaintext   |                     |
     |                       |                        |                     |
     |◄── plaintext body ────|                        |                     |
     |  (Thunderbird displays |                       |                     |
     |   normal readable mail)|                       |                     |
     |                       |                        |                     |
     |  [Send mail via SMTP] |                        |                     |
     |                       |                        |                     |
     |  SMTP AUTH ──────────►|                        |                     |
     |  user / plaintext pw  |                        |                     |
     |                       |                        |                     |
     |      3. Bridge hashes password (PBKDF2)        |                     |
     |         Translates AUTH LOGIN → AUTH PLAIN     |                     |
     |                       |                        |                     |
     |         SMTP to Postfix:587 ──────────────────────────────────────── |
     |         AUTH PLAIN user / hash                                        |
     |         Postfix → SASL server :12345 ──────────────────────────────► |
     |                                                Validates hash vs DB   |
     |◄──────────────────────────────── 235 Auth OK ──────────────────────── |
     |                       |                        |                     |
     |  DATA (message) ─────►|─── DATA verbatim ──────────────────────────► |
     |                       |    to Postfix                                 |
     |                       |    (no mail-level encrypt in bridge)          |
```

Key security property: Thunderbird operates as a completely standard IMAP/SMTP client. It does not need any plugins or special configuration. All cryptographic adaptation happens transparently in the Bridge. The mail client sees only decrypted mail and the server sees only hashed credentials.

Connection Summary

From	To	Protocol	Port	Notes
Internet	Postfix	SMTP	25	Inbound MX
Mail client	ICEMail Bridge	IMAP	143	Plain (local)
Mail client	ICEMail Bridge	IMAPS	993	TLS (local)
Mail client	ICEMail Bridge	SMTP	1587	Submission (local)
ICEMail Bridge	James	IMAPS	993	Fetch mailboxes and messages for mail client; trust-all TLS
ICEMail Server	James	IMAPS	993	Fetch mailboxes and messages for browser web mail sessions; trust-all TLS
ICEMail Bridge	Postfix	SMTP+STARTTLS	587	Mail submission from mail client
Browser	ICEMail Server	HTTPS	8282	Web app + API
ICEMail Server	Postfix	SMTP+STARTTLS	587	Send account confirmation emails
ICEMail Server	James WebAdmin	HTTP	8000	Proxy admin operations to browser (mailboxes, queues, domains)
Postfix	ICEMail Server (after-queue filter)	SMTP	10026	Postfix routes all inbound mail into the filter
ICEMail Server (after-queue filter)	Postfix	SMTP	10027	Encrypted mail re-injected back to Postfix for final delivery
Postfix	James	LMTP	24	Postfix **pushes** encrypted mail to James; James listens and does not pull
Postfix	ICEMail Server	TCP (Dovecot SASL)	12345	SMTP AUTH delegation — Postfix asks ICEMail Server to validate credentials
James	ICEMail Server	HTTPS	8282	James polls `/api/users` to synchronise user accounts